CURSO DE TECNOLOGIA EM
ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

INSTITUTO FEDERAL
DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
Ceará



ENGENHARIA DE SOFTWARE

PROF. ME. LUCIANO MARTINS

MÓDULO III
MODELOS DE PROCESSO DE SOFTWARE E METODOLOGIAS ÁGEIS



OBJETIVOS

- Modelos de processo genéricos/tradicionais
- Desenvolvimento ágil
- Metodologias de desenvolvimento ágil

RECAPITULANDO...

- Na Unidade I, vimos que:
  - Engenharia de software compreende métodos, técnicas e ferramentas que buscam controlar o desenvolvimento de software visando sua qualidade.
- Na Unidade II, vimos que:
  - O processo de software organiza as etapas envolvidas em um projeto de software, desde a sua concepção até a manuteção feita pós entrega para o cliente.



AQUECIMENTO

Um software de controle de tráfego aéreo e aplicativo móvel de rede social são desenvolvidos da mesma forma?

Por quê?

Quais as semelhanças e diferenças?



AQUECIMENTO



Software de Controle de Tráfego aéreo

- Segurança e confiabilidade são críticas.
- Falhas podem causar consequências graves.
- Necessita de forte validação.
- Conformida com normas.
- Documentação é muito importante.



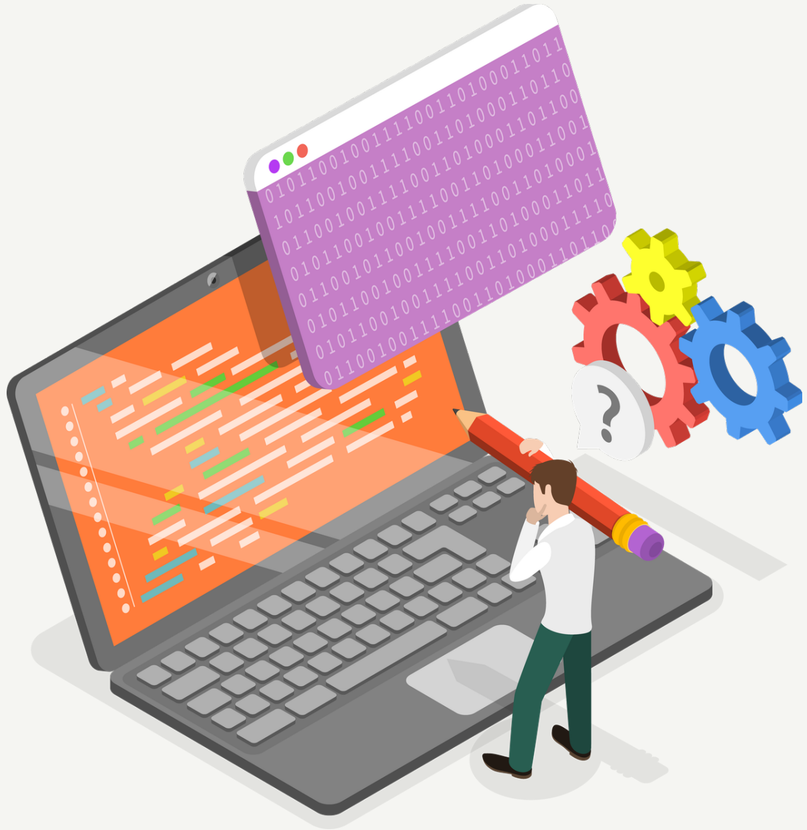
Aplicativo móvel de rede social

- Mudanças frequentes.
- Grande interação com usuário.
- Necessidade de incorporar mudanças rapidamente.
- Prioridade pode ser a rapidez nas entregas.

Softwares diferentes precisam de diferentes processos de desenvolvimento!

INTRODUÇÃO

- A Engenharia de software busca organizar e apoiar as etapas do desenvolvimento de software com foco na qualidade.
- O Ciclo de Vida do Desenvolvimento de Software organiza o trabalho em atividades (Especificação, Desenvolvimento, Validação e Evolução).
- Entretanto, existem diferentes formas de organizar e representar os processos de software: **modelos de processo de software.**



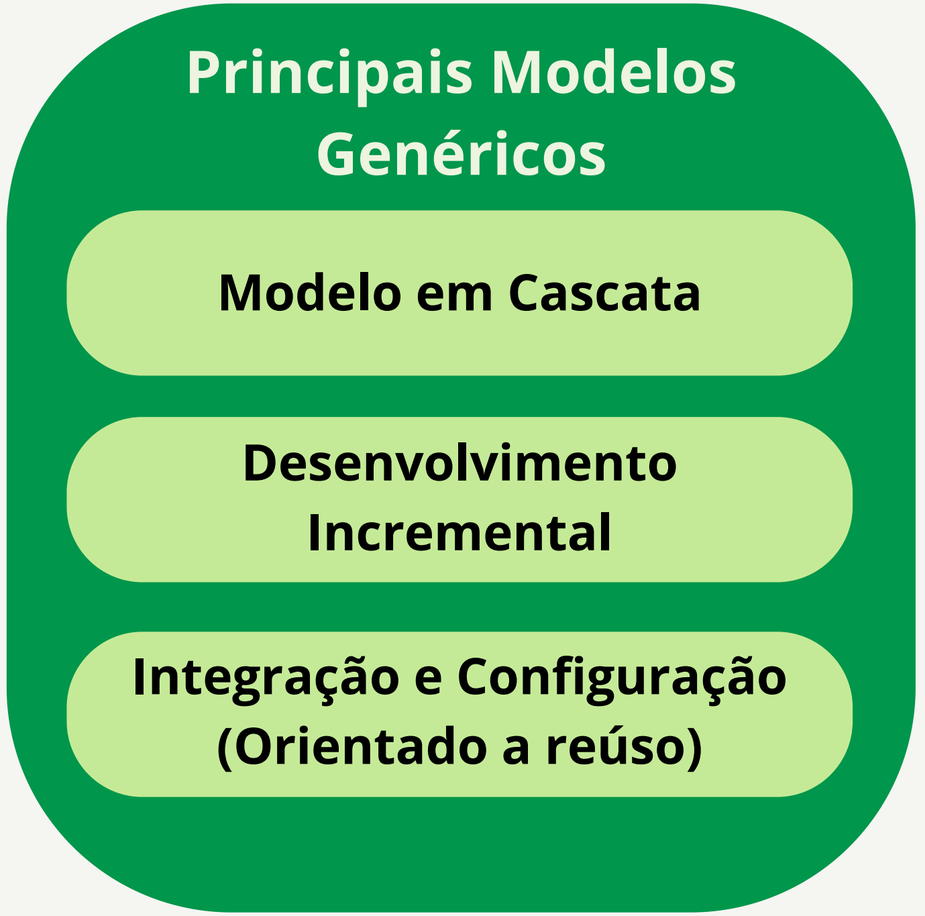
MODELO DE PROCESSO

- Forma de organizar e estruturar as atividades, responsabilidades e entregas envolvidas no desenvolvimento de um software.
- Um modelo define:
  - Quais atividades devem ser realizadas e sua ordem
  - A relação entre as atividades
  - Quando e quais serão as entregas
  - Como mudanças e feedbacks são incluídos no decorrer do projeto.
- O modelo de processo guia o trabalho da equipe e o andamento do projeto de software.



MODELOS GENÉRICOS

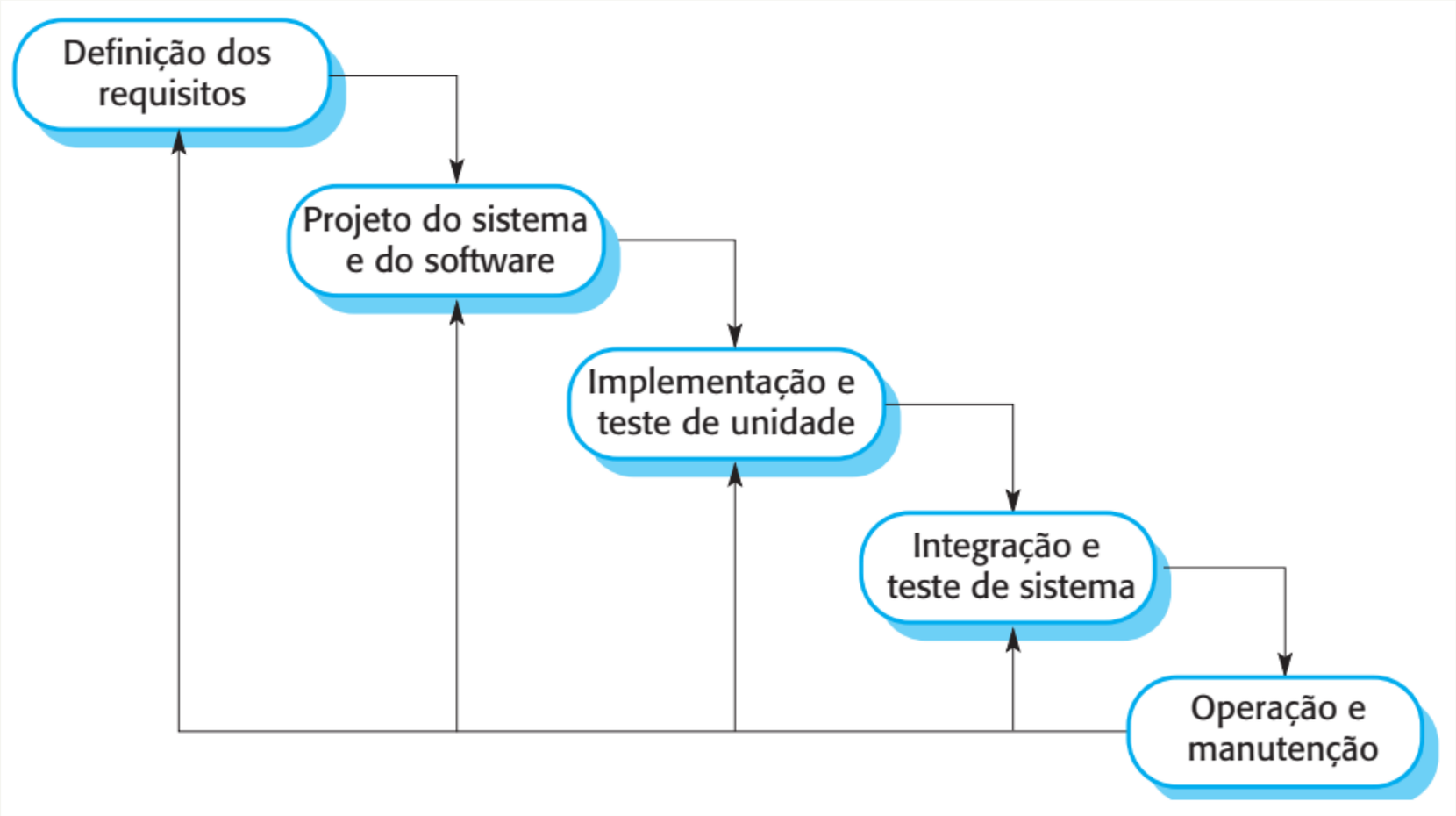
- Também conhecidos como paradigmas de processo.
- São descrições mais gerais e abstratas dos processos de software, que podem ser utilzadas para explicar diferentes abordagens de desenvolvimento.
- Podem ser vistos como estruturas de processo que podem ser adaptadas e ampliadas para criar processos de engenharia mais específicos.



MODELO EM CASCATA

- Primeiro modelo de processo publicado.
- Baseia-se no modelo de construção de grandes sistemas militares.
- Nele, o processo de desenvolvimento é apresentado como uma série de estágios.
- É um exemplo de **modelo de processo dirigido por plano**.
- Cada fase resulta em um ou mais documentos que passam por aprovação.
- A fase seguinte não deve começar até que a anterior termine.

MODELO EM CASCATA



Fonte: Sommerville(2018)

MODELO EM CASCATA

1. **Análise e definição de requisitos:** usuários são consultados para estabelecer os serviços, as restrições e as metas do sistema, que gerarão a especificação do sistema.
2. **Projeto do sistema e do software:** requisitos são repartidos em de hardware e de software e uma arquitetura global do sistema é estabelecida. São descritas as abstrações fundamentais das partes do sistema e seus relacionamentos.
3. **Implementação e teste de unidade:** projeto de software é realizado como um conjunto de unidades de programa e cada unidade passa por verificação.
4. **Integração e teste de sistema:** partes do programa são integradas e testadas como um sistema completo, verificando a conformidade com requisitos. Em seguida, sistema é entregue ao cliente.
5. **Operação e manutenção:** sistema é instalado e colocado em uso. Manutenção compreende correção de erros, implementação de melhorias e aperfeiçoamento de serviços com base em novos requisitos.

MODELO EM CASCATA



Problemas:

- **Processo de software na prática não é linear**: feedbacks e descobertas de problemas e requisitos entre as fases podem demandar alterações e consequentemente validações.
- O congelamento de requisitos para evitar mudanças pode gerar inadequação do sistema às necessidades do usuário.

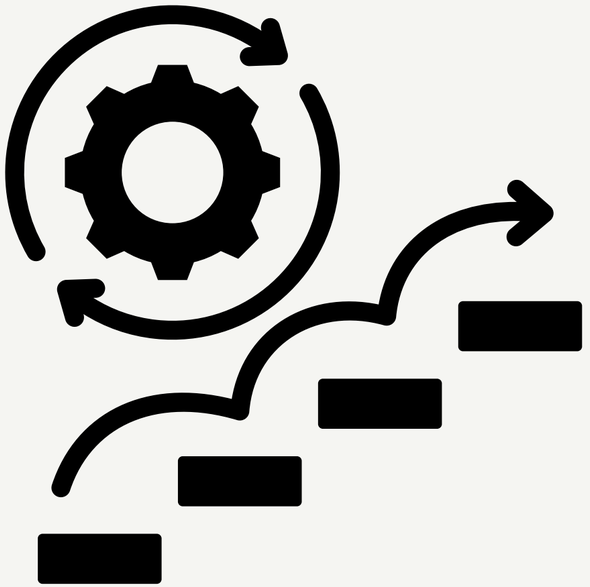
Quando usar o modelo em cascata?

- Desenvolvimento de sistemas embarcados
- Desenvolvimento de sistemas críticos
- Grandes sistemas de software que fazem parte de outros sistemas maiores desenvolvidos por várias empresas.

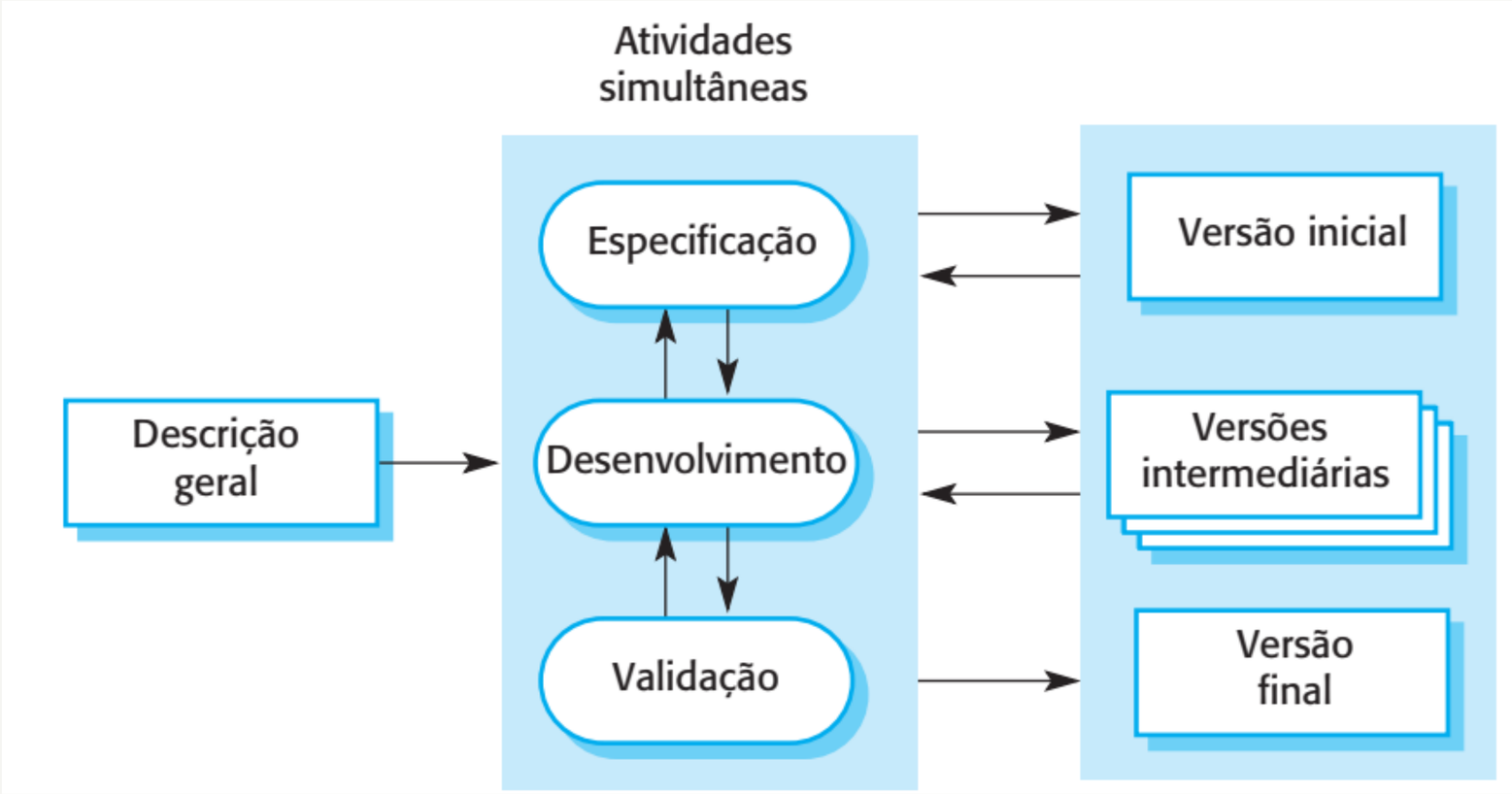


DESENVOLVIMENTO INCREMENTAL

- Baseia-se na ideia de desenvolver uma implementação inicial, obter feedback dos *stakeholders* e fazer o software evoluir por meio de versões, até a conclusão do sistema.
- As atividades de especificação, desenvolvimento e validação são intercaladas, com feedbacks rápidos ao longo de todas elas.
- Pode se tanto uma abordagem dirigida por planos quanto ágil.
- Melhor para sistemas cujos requisitos estão propensos a mudar durante o processo de desenvolvimento.



DESENVOLVIMENTO INCREMENTAL



Fonte: Sommerville(2018)

DESENVOLVIMENTO INCREMENTAL



Vantagens

- O custo de implementação de mudanças nos requisitos é reduzido.
- É mais fácil obter feedback do cliente sobre o trabalho de desenvolvimento.
- Entrega e implementação antecipada para o cliente são possíveis antes do sistema ser finalizado (entrega de incremental).

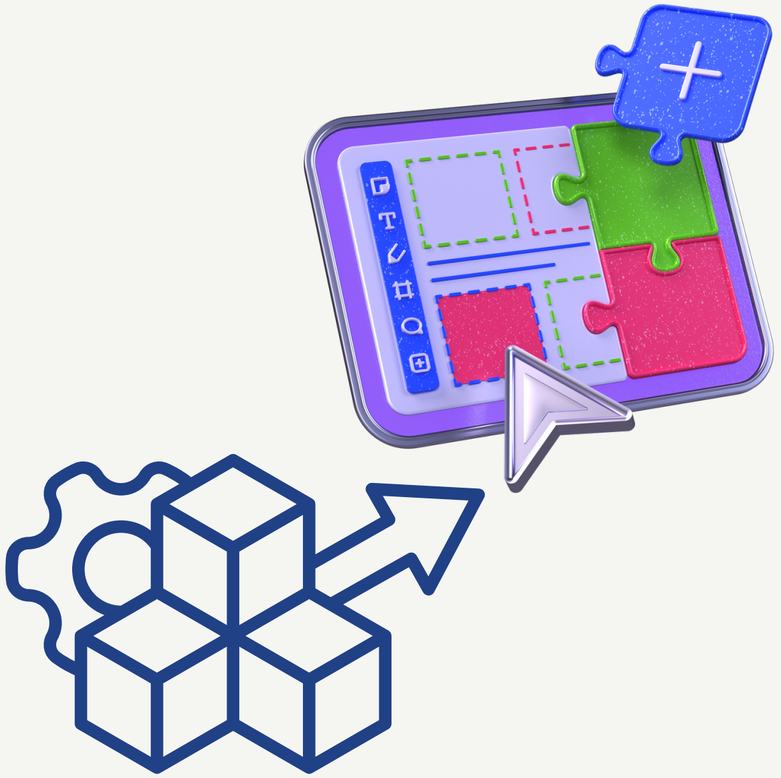


Problemas

- O processo não é visível, devido a dificuldade de medir o processo, dada a rapidez no desenvolvimento.
- A estrutura do sistema tende a se degradar à medida que os incrementos são adicionados.
- Problemas se tornam mais críticos para sistemas grandes, complexos e de vida longa.
- Dificuldade de alinhamento com procedimentos burocráticos.

INTEGRAÇÃO E CONFIGURAÇÃO

- Também chamado de **Modelo Orientado a Reúso**.
- A maioria dos projetos utiliza reúso de software: códigos já prontos são procurados, modificados e integradoa a um novo código.
- A partir dos anos 2000, os processos de software baseados em reúso começaram a ser amplamente utilizados.
- Abordagem orientada a reúso contam com bases de componentes reusáveis e um framework para integração.



INTEGRAÇÃO E CONFIGURAÇÃO

Três tipos de componentes muito reusádos:

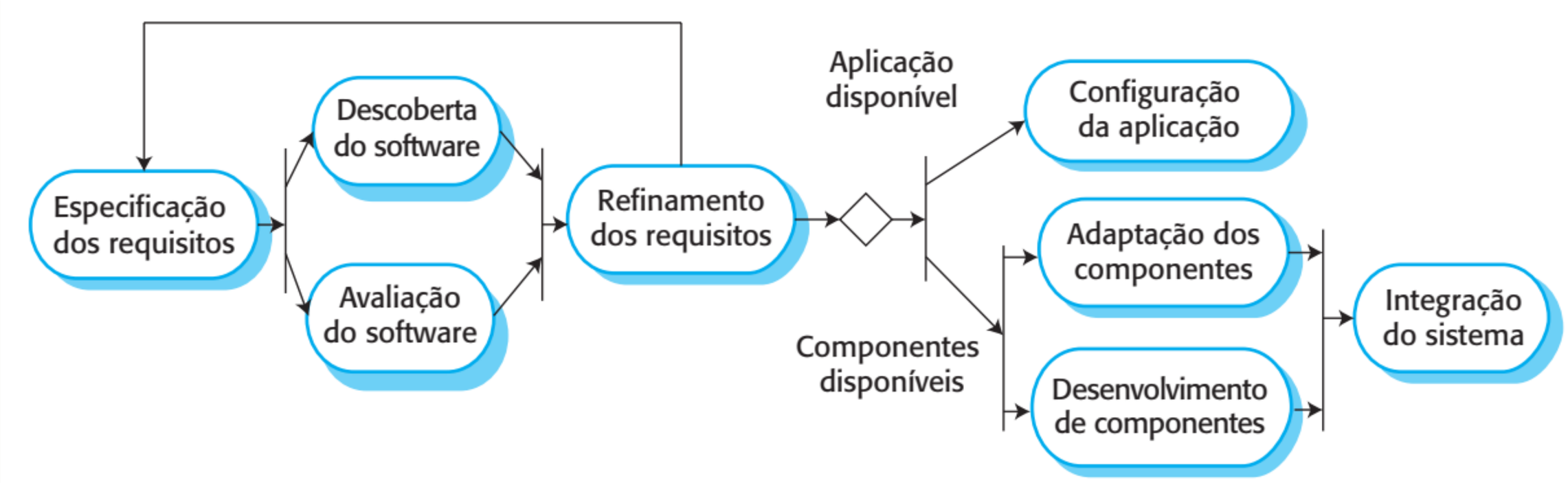
Sistemas de aplicação stand-alone configurados para ambientes específicos
Exemplo: sistema de gestão escolar pronto, que é configurado para uma determina instituição, ajustando cursos, turmas, relatórios etc.

Coleções de objetos desenvolvidos como um pacote a ser integrado em um framework
Exemplo: uma biblioteca de autenticação desenvolvida para uma aplicação que pode ser integrada a um framework como o Spring, fornecendo as funcionalidades necessárias.

Web services densenvolvidos conforme padrões e que estão disponíveis para uso remoto na internet
Exemplo: uma plataforma que utiliza serviço de pagamento de uma empresa externa via API.

INTEGRAÇÃO E CONFIGURAÇÃO

Engenharia de Software Orientada a Reúso



Fonte: Sommerville(2018)

INTEGRAÇÃO E CONFIGURAÇÃO

Engenharia de Software Orientada a Reúso

1. Os **requisitos iniciais do sistema são propostos**, incluindo descrições breves dos requisitos essenciais e das características desejáveis para o sistema.
2. Com base nos requisitos descritos, **componente**s e sistemas que fornecem as funcionalidades **são buscados e avaliados**, a fim de verificar adequação.
3. Os **requisitos são modificados de modo a refletir os componentes disponíveis** e a especificação do sistema é redefinida. Pode-se fazer uma nova busca de componentes.
4. Caso uma aplicação que satisfaça os requisitos esteja disponível, ela pode ser configurada para uso sem que um novo sistema seja desenvolvido.
5. **Caso não haja aplicação disponível**, os **componentes reúsaveis podem ser modificados** ou novos componentes podem ser desenvidos para posterior integração.

INTEGRAÇÃO E CONFIGURAÇÃO



Vantagens

- Reduz a quantidade de software a ser desenvolvido.
- Diminui custos e riscos.
- Leva a uma entrega mais rápida.



Problemas

- Necessidade de concessões em relação aos requisitos.
- Potencial insatisfação das reais necessidades.
- Dificuldade de controle da evolução do sistema.

BORA TESTAR?!

(A) Modelo em Cascata
(B) Modelo Incremental
(C) Modelo Baseado em Reúso

( ) O sistema é desenvolvido em partes, permitindo entregas parciais e feedback do usuário ao longo do tempo.
( ) Todas as etapas do desenvolvimento são realizadas de forma sequencial, com pouca ou nenhuma sobreposição entre elas.
( ) Mudanças são difíceis de serem incorporadas após o início do desenvolvimento.
( ) É comum o uso de componentes de código já existente, o que pode aumentar a produtividade, mas gerar dependência de terceiros.
( ) O cliente só vê o sistema funcionando ao final do desenvolvimento.
( ) O desenvolvimento utiliza bibliotecas, frameworks ou componentes prontos para acelerar a construção do sistema.
( ) Funciona muito bem para projetos com requisitos muito claros e com pouca probabilidade de mudanças.

MODELOS DE PROCESSOS - RESUMO

INSTITUTO FEDERAL
DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
Ceará

Modelo de Processo	Quando usar	Quando evitar
Cascata	• Requisitos são bem definidos. • Poucas mudanças são esperadas. • Cliente tem clareza do que quer.	• Requisitos não são claros. • Cliente precisar de versões parciais do sistema. • Sistema é muito “inovador”.
Incremental	• Sistema puder ser dividido em partes. • Versões parciais são importantes. • Mudanças são esperadas. • Cliente precisar ser envolvido no processo.	• Sistema não puder ser facilmente dividido. • Cliente não puder participar do processo. • Existir forte dependência entre as partes do sistema.
Orientado a reúso	• Existir componentes disponíveis. • Problema é comum e já possui muitas soluções. • Equipe conhece os componentes. • Tempo necessita ser curto.	• Não existir componentes adequados. • O sistema exige soluções muito específicas. • Equipe não domina os componentes. • Existem restrições severas de segurança e dependência externa.

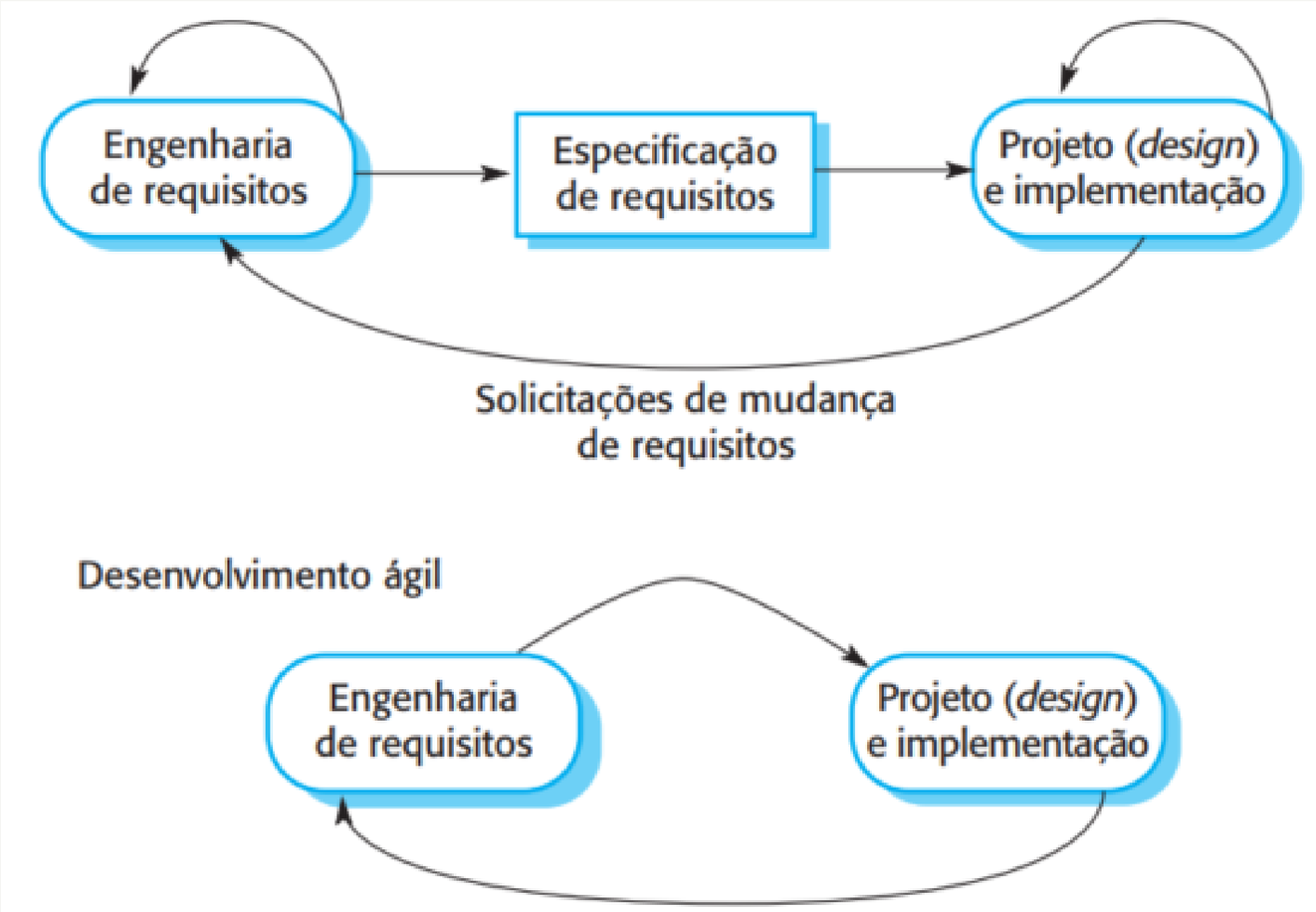
DESENVOLVIMENTO ÁGIL DE SOFTWARE

CONTEXTO...

- É necessário que o desenvolvimento de software acompanhe o ritmo dinâmico dos ambientes.
- Consequência: empresas estão dispostas a negociar a qualidade e comprometer requisitos, a fim de que obtenham o software mais rápido.
- Modelos de processo dirigidos por planos são mais lentos. Por quê?
- Solução: Metodologias Ágeis



DESENVOLVIMENTO DIRIGIDO POR PLANOS X DESENVOLVIMENTO ÁGIL



Fonte: Sommerville(2018)

MÉTODOS ÁGEIS

- Surgiram no final da década de 1990 em resposta à insatisfação com a abordagem pesada dirigida por planos dominante na época.
- Têm como foco principal **entregar o mais rapidamente um software funcional** para o cliente, que poderá propor a inclusão de requisitos novos ou modificados a cada iteração.
- O objetivo é reduzir a burocracia do processo e focar nas atividades relacionadas diretamente ao desenvolvimento.
- Filosofia por trás dos métodos ágeis está registrada no documento chamado **Manifesto Ágil**.



MANIFESTO ÁGIL



Estamos descobrindo maneiras melhores de desenvolver software, fazendo-o nós mesmos e ajudando outros a fazerem o mesmo. Através deste trabalho, passamos a valorizar:

Indivíduos e interações *mais que processos e ferramentas*
Software em funcionamento *mais que documentação abrangente*
Colaboração com o cliente *mais que negociação de contratos*
Responder a mudanças *mais que seguir um plano*

Ou seja, mesmo havendo valor nos itens à direita, valorizamos mais os itens à esquerda.

Fonte: https://agilemanifesto.org/iso/ptbr/manifesto.html

MÉTODOS ÁGEIS

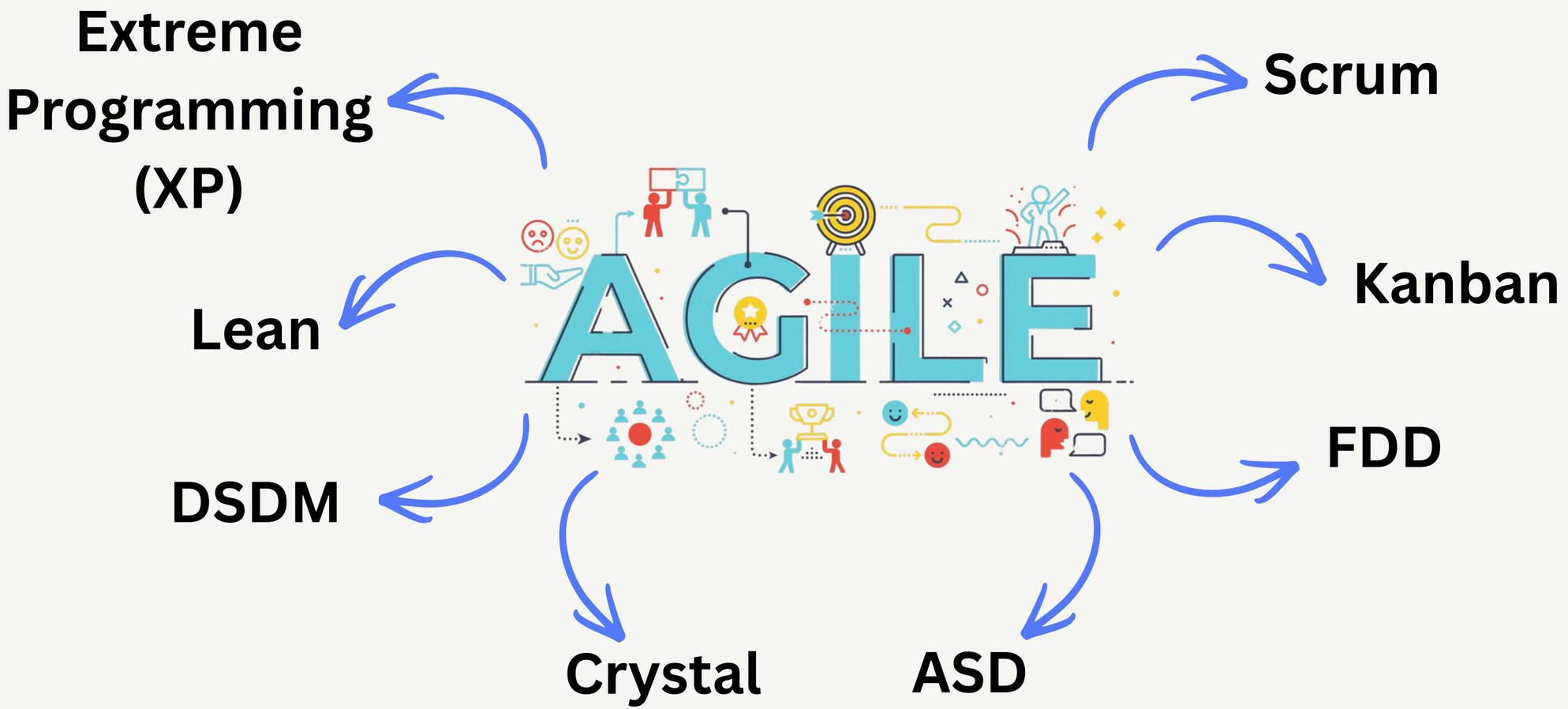
Princípios

INSTITUTO FEDERAL
DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
Ceará

Princípio	Descrição
Envolvimento do cliente	Os clientes devem ser envolvidos em todo o processo de desenvolvimento. Seu papel é fornecer e priorizar novos requisitos de sistema e avaliar as iterações do sistema.
Acolher as mudanças	Tenha em mente que os requisitos do sistema mudam e, portanto, deve-se projetar o sistema para acomodar essas mudanças.
Entrega incremental	O software é desenvolvido em incrementos, e o cliente especifica os requisitos incluídos em cada um deles.
Manter a simplicidade	Deve-se ter como foco a simplicidade, tanto do software que está sendo desenvolvido quanto do processo de desenvolvimento. Sempre que possível, trabalhe ativamente para eliminar a complexidade do sistema.
Pessoas, não processos	As habilidades do time de desenvolvimento devem ser reconhecidas e aproveitadas da melhor maneira possível. Seus membros devem ter liberdade para desenvolver seu modo próprio de trabalhar sem se prender a processos determinados.

Fonte: Sommerville(2018)

MÉTODOS ÁGEIS



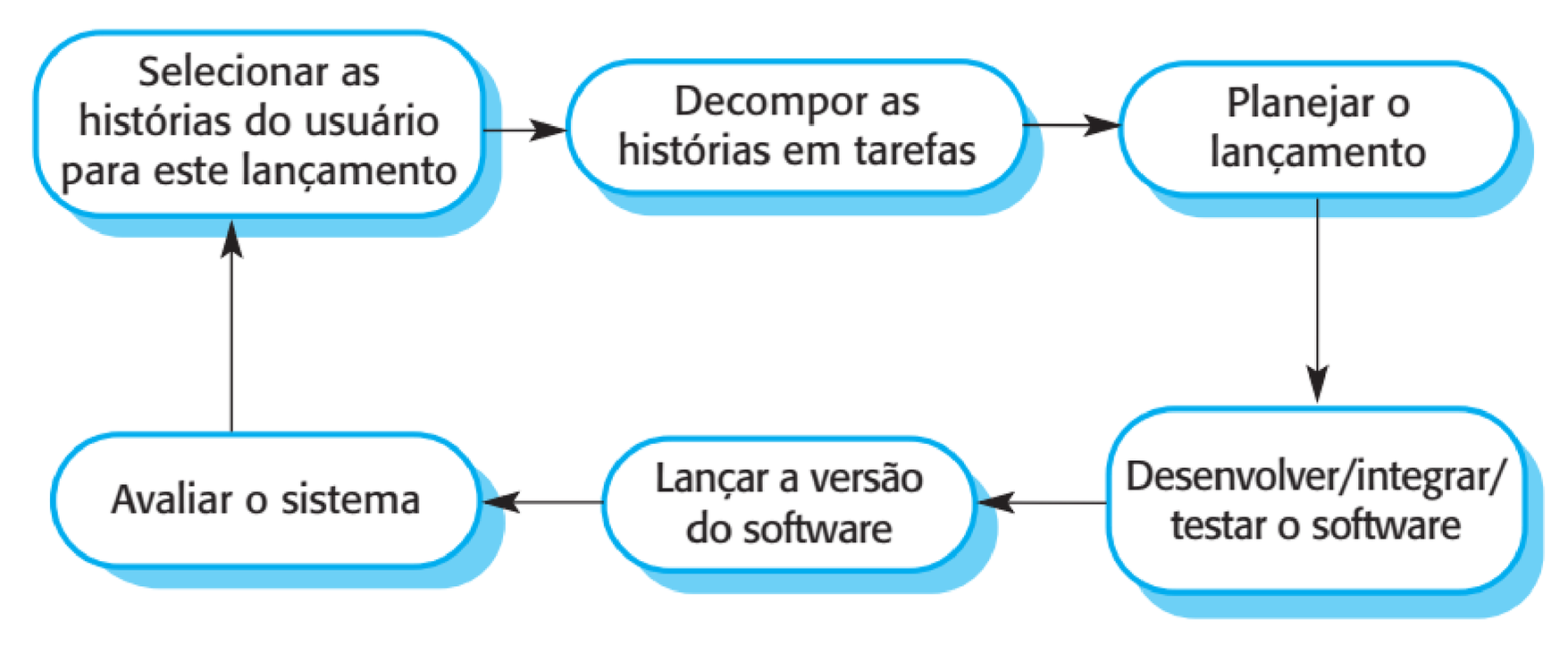
EXTREME PROGRAMMING (XP)

EXTREME PROGRAMMING (XP)

- Termo cunhado em 1998 por Kent Beck, foi uma das primeiras abordagens de desenvolvimento ágil.
- Muito importante para a época para a mundança de cultura no desenvolvimento.
- Nela, os requisitos são expressos por meio de cenários, denominados **histórias do usuários**, implementados como uma série de tarefas.
- Utiliza a **programação em pares**: Os programadores trabalham em pares e desenvolvem testes para cada tarefa antes de escreverem o código.
- Os testes devem ser executados com sucesso quando o novo código é integrado ao sistema.
- O intervalo entre os incrementos é curto e é denominado **release**.

EXTREME PROGRAMMING (XP)

- Ciclo de lançamento (release) de uma versão da XP.



Fonte: Sommerville(2018)

PRINCÍPIOS/PRÁTICAS XP

INSTITUTO FEDERAL
DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
Ceará

1. **Propriedade coletiva:** Os pares de programadores trabalham em todas as partes do sistema e têm responsabilidade sobre todo o código.
2. **Integração contínua:** Quando o trabalho de uma tarefa é concluído, ele é integrado ao sistema, que deve ser testado com sucesso em seguida.
3. **Planejamento incremental:** Requisitos registrados em cartões de histórias, decompostos em tarefas, e escolhidos de acordo com o tempo disponível.
4. **Representando do cliente:** Um representante do cliente deve estar disponível em tempo integral para o time de programação.
5. **Programação em pares:** Os desenvolvedores trabalham em pares, revisando o trabalho um do outro e oferecendo suporte necessário.
6. **Refatoração:** Os desenvolvedores devem refatorar o código continuamente assim que forem encontradas possibilidades de melhorias, mantendo o código simples e de fácil manutenção.
7. **Projeto simples:** O projeto do sistema deve incluir apenas o suficiente para satisfazer os requisitos atuais e nada mais.
8. **Lançamentos pequenos:** Primeiro deve ser desenvolvido um conjunto mínimo de funcionalidades e as demais funcionalidades vão sendo agregadas de forma incremental por lançamentos frequentes.
9. **Ritmo sustentável:** Deve-se manter um ritmo de trabalho consistente sem prejudicar a qualidade do código.
10. **Test-first:** Os testes de um novo pedaço de funcionalidade devem ser desenvolvidos antes dela própria.

HISTÓRIAS DO USUÁRIO NA XP

- **Objetivo**: formar cenários de uso baseados nas experiências dos usuários do sistema, no intuito de coletar os requisitos de forma integrada ao desenvolvimento.

- **Ponto positivo:** envolvimento do cliente no processo é mais fácil.
- **Ponto negativo:** pode afetar a completude dos requisitos do sistema.

Utilização

- Resumo de um cenário é registrado em um **cartão de história**.
- As versões do sistema são desenvolvidas com base na implementação dos cenários definidos nas histórias dos usuários.
  - Cada cartão de história é decomposto em tarefas pelo time de desenvolvimento.
  - O time estima o esforço e os recursos necessários para implementar cada história.
  - Quem define a prioridade de implantação a cada versão é o cliente.



HISTÓRIAS DO USUÁRIO NA XP

INSTITUTO FEDERAL
DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
Ceará

Prescrição de medicação

Kate é uma médica que deseja prescrever medicação para um paciente que frequenta sua clínica. O registro do paciente já é exibido em seu computador, então ela deve clicar no campo de medicação e selecionar 'medicação atual', 'nova medicação' ou 'substâncias'.

Se escolher 'medicação atual', o sistema pedirá para que ela verifique a dose; se quiser mudá-la, Kate fornecerá a nova dose e depois deverá confirmar a prescrição.

Se escolher 'nova medicação', o sistema presumirá que ela sabe qual medicação prescrever. Ao digitar as primeiras letras da medicação, o sistema exibirá uma lista de possíveis medicamentos que começam com essas letras. Ao escolher a medicação necessária, o sistema responderá pedindo que ela confirme se a medicação escolhida está correta. Ela deverá fornecer a dose e depois confirmar a prescrição.

Se ela escolher 'substâncias', o sistema exibirá uma caixa de busca para a substância aprovada, e então ela poderá pesquisar o medicamento que deseja. Feita a busca, lhe será solicitado que confirme se o medicamento selecionado está correto. Ela deverá fornecer a dose e depois confirmar a prescrição.

O sistema sempre verificará se a dose está dentro da faixa aprovada. Se não estiver, será pedido a Kate que faça uma alteração.

Depois de confirmar a prescrição, ela será exibida para conferência. Kate deverá clicar em 'OK' ou em 'Alterar'. Se 'OK' for selecionado, a prescrição será registrada no banco de dados de auditoria. Se clicar em 'Alterar', o processo de 'Prescrição de medicação' é executado novamente.

HISTÓRIAS DO USUÁRIO NA XP

INSTITUTO FEDERAL
DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
Ceará

Prescrição de medicação

Kate é uma médica que deseja prescrever medicação para um paciente que frequenta sua clínica. O registro do paciente já é exibido em seu computador, então ela deve clicar no campo de medicação e selecionar 'medicação atual', 'nova medicação' ou 'substâncias'.

Se escolher 'medicação atual', o sistema pedirá para que ela verifique a dose; se quiser mudá-la, Kate fornecerá a nova dose e depois deverá confirmar a prescrição.

Se escolher 'nova medicação', o sistema presumirá que ela sabe qual medicação prescrever. Ao digitar as primeiras letras da medicação, o sistema exibirá uma lista de possíveis medicamentos que começam com essas letras. Ao escolher a medicação necessária, o sistema responderá pedindo que ela confirme se a medicação escolhida está correta. Ela deverá fornecer a dose e depois confirmar a prescrição.

Se ela escolher 'substâncias', o sistema exibirá uma caixa de busca para a substância aprovada, e então ela poderá pesquisar o medicamento que deseja. Feita a busca, lhe será solicitado que confirme se o medicamento selecionado está correto. Ela deverá fornecer a dose e depois confirmar a prescrição.

O sistema sempre verificará se a dose está dentro da faixa aprovada. Se não estiver, será pedido a Kate que faça uma alteração.

Depois de confirmar a prescrição, ela será exibida para conferência. Kate deverá clicar em 'OK' ou em 'Alterar'. Se 'OK' for selecionado, a prescrição será registrada no banco de dados de auditoria. Se clicar em 'Alterar', o processo de 'Prescrição de medicação' é executado novamente.

Tarefa 1: Mudar a dose do medicamento prescrito

Tarefa 2: Seleção de substância

Tarefa 3: Verificação da dose

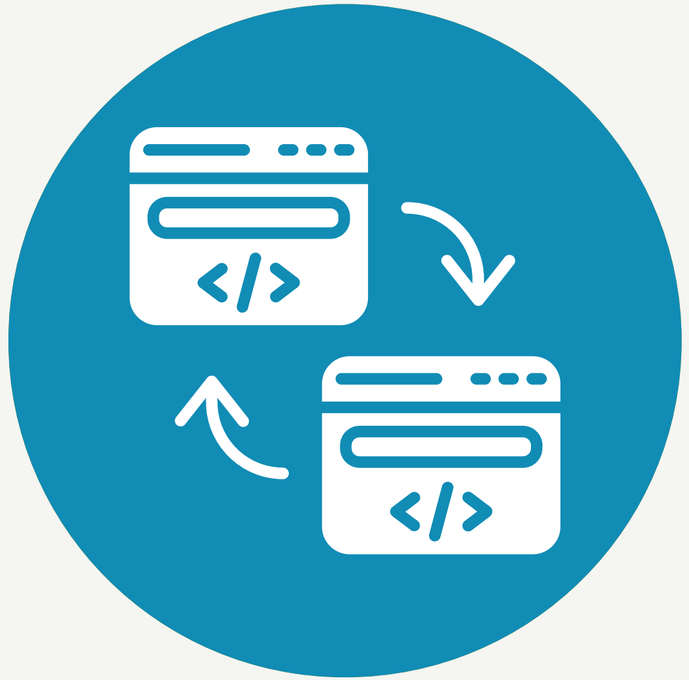
A verificação da dose é uma precaução de segurança para conferir se o médico não prescreveu uma dose perigosamente pequena ou grande.

Usando a identificação da substância para o nome genérico do medicamento, procure a substância e recupere a dose máxima e a mínima recomendadas.

Confira a dose prescrita em relação ao máximo e ao mínimo. Se estiver fora da faixa, emita uma mensagem de erro dizendo que a dose é alta ou baixa demais. Se estiver dentro da faixa, habilite o botão "Confirmar".

REFATORAÇÃO NA XP

- O time de programação deve buscar possíveis melhorias no software e implementá-las imediatamente.
- Colabora para melhorar a estrutura e clareza do software, evitando a deterioração que pode ser resultante do desenvolvimento incremental.
- Exemplos: reorganização de hierarquia entre classes, remoção de códigos duplicados, renomeação de atributos etc.
- Pressão por agilidade pode comprometer a refatoração.



DESENVOLVIMENTO *TEST-FIRST*

INSTITUTO FEDERAL
DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
Ceará

- Desenvolvimento com testes *a priori*.

- Metodologia de testes proposta na XP com as características:

1. Os testes são desenvolvidos antes do código.
2. Testes incrementais criados a partir dos cenários.
3. Envolvimento do usuário na criação e validação dos testes.
4. Uso de frameworks automatizados.

- Os testes são escritos como componentes executáveis antes de a tarefa ser implementada, devendo simular entradas e conferir se resultados satisfazem especificações de saídas.
- Problemas:
  - Garantir a cobertura dos testes.
  - Adesão dos programadores.
  - Dificuldade de escrever alguns testes.

DESENVOLVIMENTO *TEST-FIRST*

Teste 4: Conferir a dose

Entrada:

1. Um número em mg representando uma única dose de medicamento.
2. Um número representando a quantidade de doses únicas por dia.

Testes:

1. Teste das entradas em que a dose única está correta, mas a frequência é alta demais.
2. Teste das entradas em que a dose única é alta demais e baixa demais.
3. Teste das entradas em que a dose única * frequência é alta demais e baixa demais.
4. Teste das entradas em que a dose única * frequência está no intervalo permitido.

Saída:

'OK' ou mensagem de erro indicando que a dose está fora do intervalo seguro.

PROGRAMAÇÃO EM PARES

- Cada dupla de programadores senta-se diante do mesmo computador para desenvolver o software.
- Pares são criados dinamicamente para que todos os membros do time possam trabalhar juntos durante o processo.

- Vantagens:
  - Apoia a ideia de propriedade e responsabilidade coletiva pelo sistema.
  - Age como processo de revisão informal.
  - Incentiva a refatoração para melhorar a estrutura do software.



SCRUM



SCRUM

- Método ágil criado com o **objetivo de gerenciar os processos profissionais de desenvolvimento de software**, de modo a fazer o melhor uso dos recursos e do tempo.
- Alinha-se aos princípios do manifesto ágil, concentrando-se em proporcionar um arcabouço para a organização ágil dos projetos, mas não foca em impor práticas de desenvolvimento específicas.
- Pode ser integrado de modo mais fácil às práticas atuais das empresas.
- Tornou-se uma das principais abordagens utilizadas no desenvolvimento.



PAPÉIS SCRUM

INSTITUTO FEDERAL
DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
Ceará



Product Owner (PO)

- Responsável por definir as funcionalidades do produto, as datas de lançamento e o conteúdo.
- Deve definir, priorizar e ajustar as funcionalidades.
- Aceita ou rejeita as versões apresentadas.



Scrum Master

- Cuida da gerência do projeto.
- Funciona como meio de comunicação entre o PO e a equipe, eliminando distrações externas dos desenvolvedores.
- Tem a função de remober obstáculos do projeto.



Equipe de desenvolvimento

- Formada por 5 a 9 pessoas com múltiplas funções (programadores, testadores, designers etc.).
- É auto-gerenciável e possui alocação em tempo integral.

ARTEFATOS SCRUM

INSTITUTO FEDERAL
DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
Ceará

Product Backlog

- Uma lista de itens como características do produto, Requisitos e melhorias a serem trabalhadas pelo time Scrum, definidos pelo PO e com prioridades.
- Pode ser originado de um documento de requisitos, histórias do usuário ou alguma descrição do software.

Sprint Backlog

- Trabalho selecionado para ser desenvolvido dentro de um sprint, levando em conta o backlog do produto, as capacidades da equipe e o andamento do projeto.

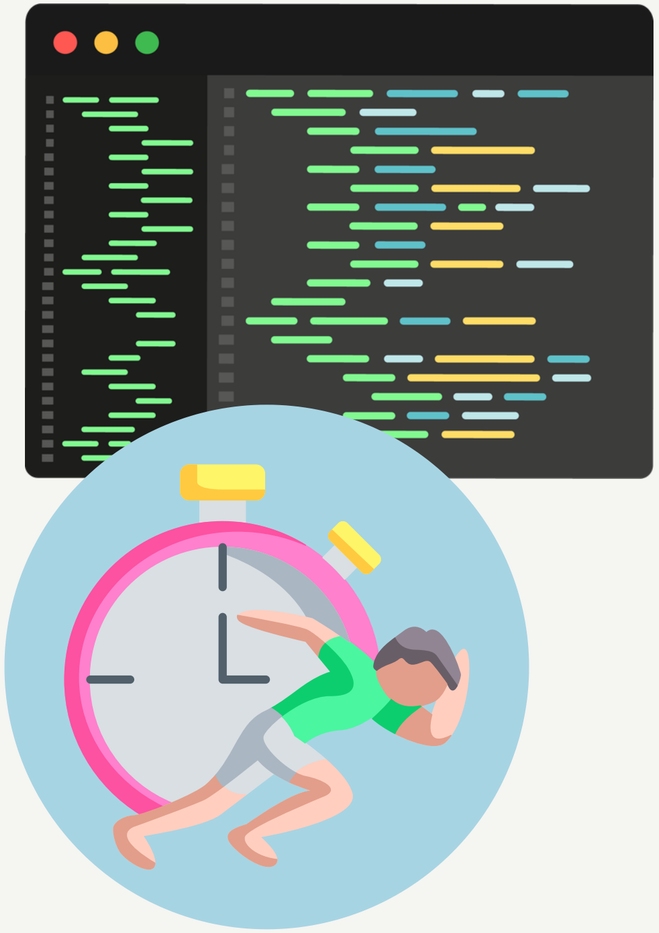
Incremento/Entrega

- Produto funcional entregue ao final de um ciclo de trabalho (*sprint*), o qual é avaliado pelo PO, que deve validar e/ou propor mudanças.



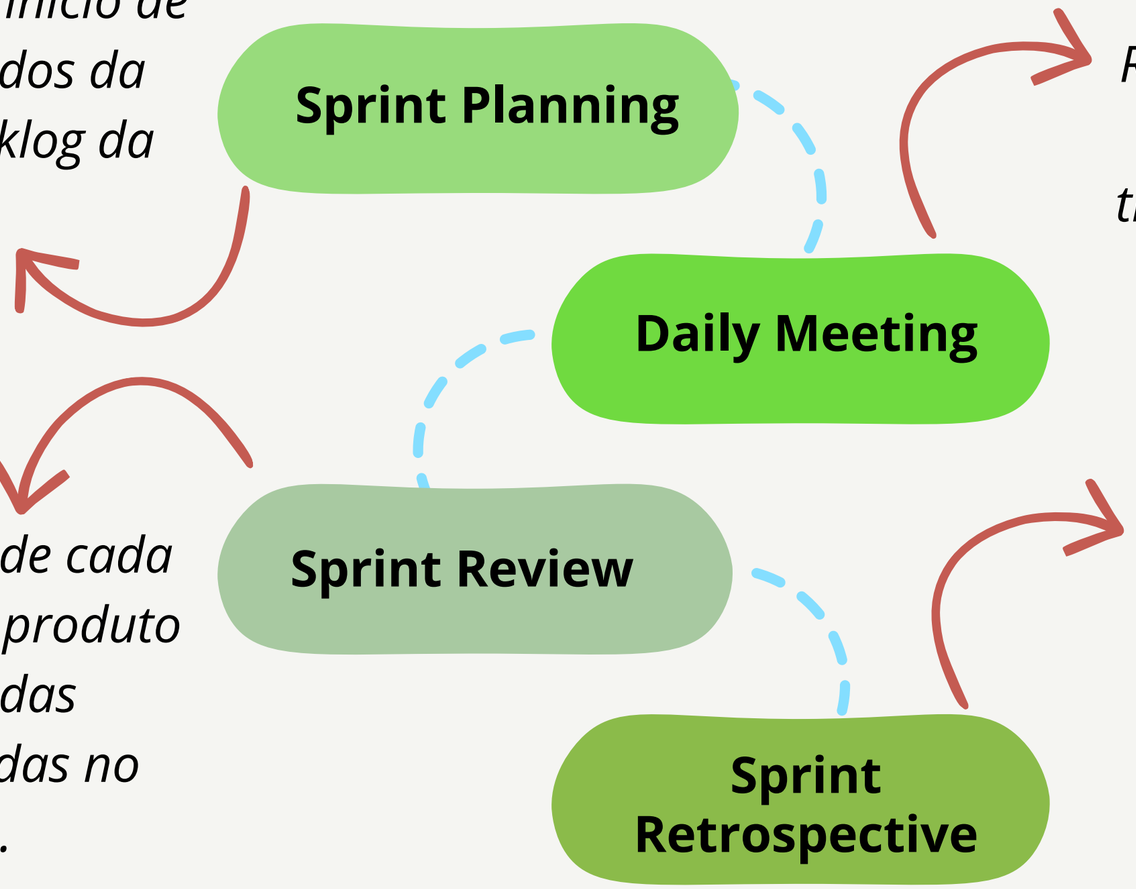
SPRINTS NO SCRUM

- Sprint é uma **unidade de planejamento** na qual o trabalho a ser feito é avaliado, os recursos são selecionados e o software é desenvolvido e testado.
- Ao final do sprint, uma funcionalidade completa é entregue.
- Tem um curto período de tempo (2 a 4 semanas).
- Leva em consideração o backlog do produto e as prioridades apresentadas pelo PO.
- No início do sprint é selecionado o conjunto de funcionalidades que serão trabalhadas (*backlog do sprint*).
- Alterações nos incrementos ficam fora do sprint.
- São feitas reuniões diárias rápidas para avaliar o trabalho e realizar readequações necessárias.



CERIMÔNIAS DO SCRUM

Reunião realizada antes do início de uma sprint envolvendo todos da equipe, para definir o backlog da sprint.

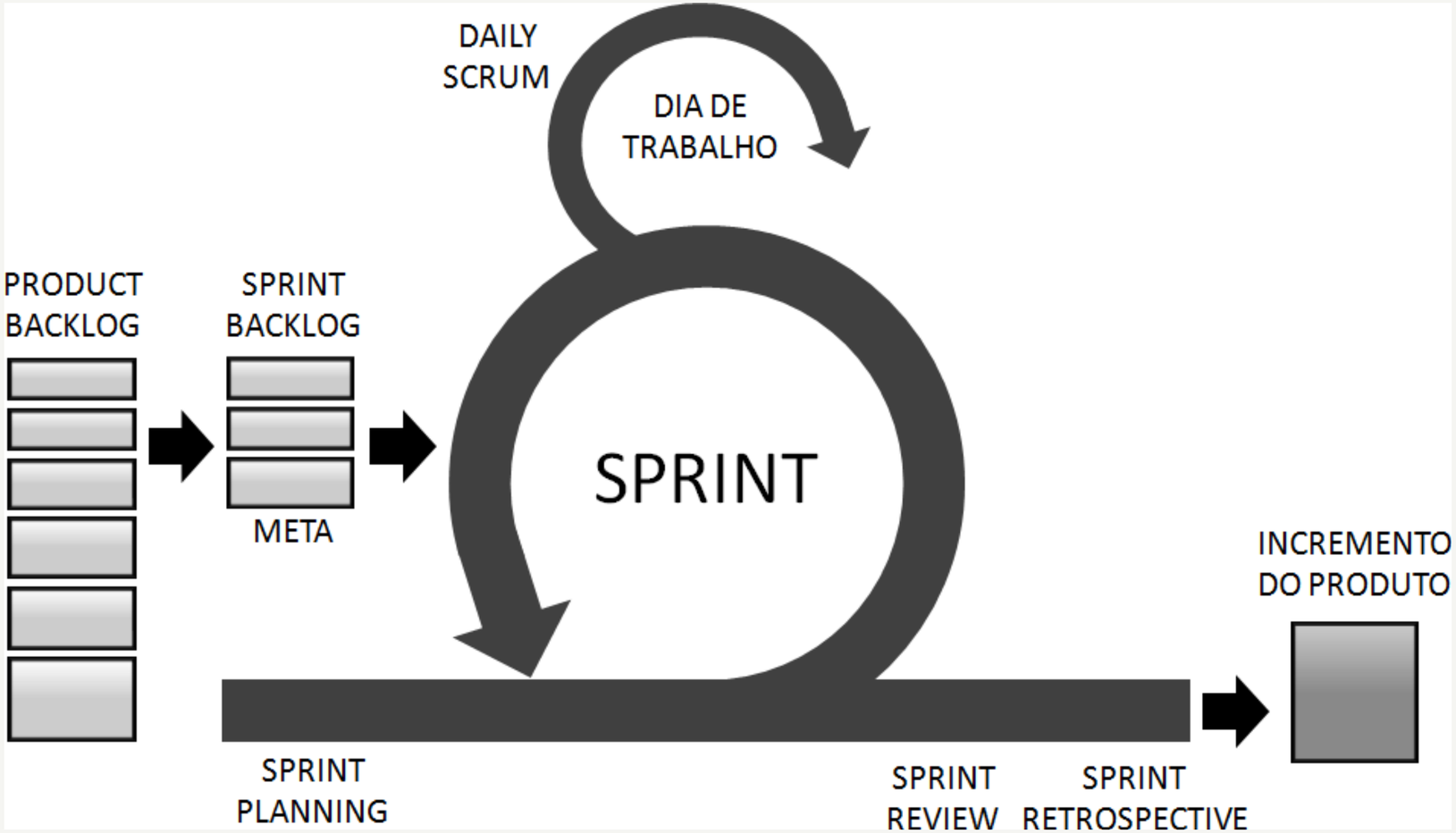


Reunião diária de curta duração onde cada membro expõe o trabalho feito no dia anterior e o que será feito no dia.

Reunião realizada ao final de cada sprint na qual é avaliado o produto resultante e são coletadas mudanças a serem inseridas no backlog do produto.

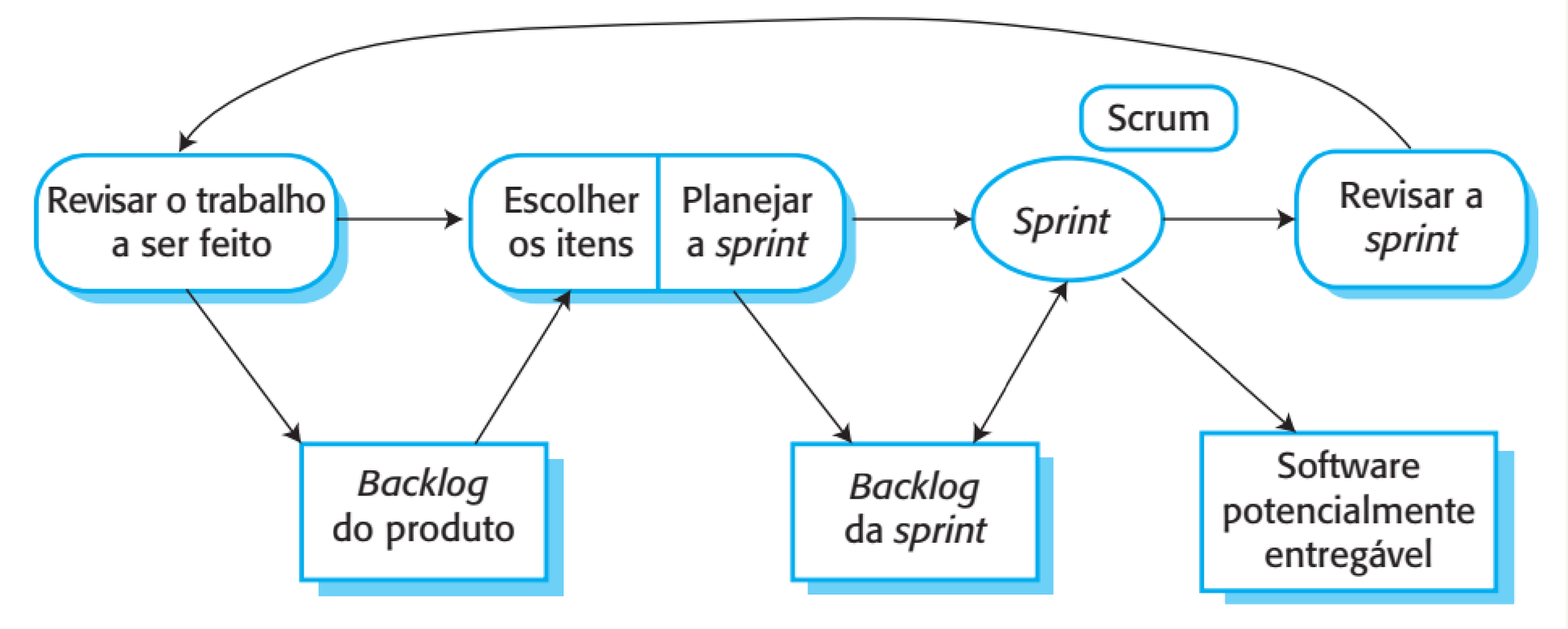
Reunião final do sprint na qual é feito o feedback do processo e identificadas necessidades de mudanças.

SPRINTS SCRUM

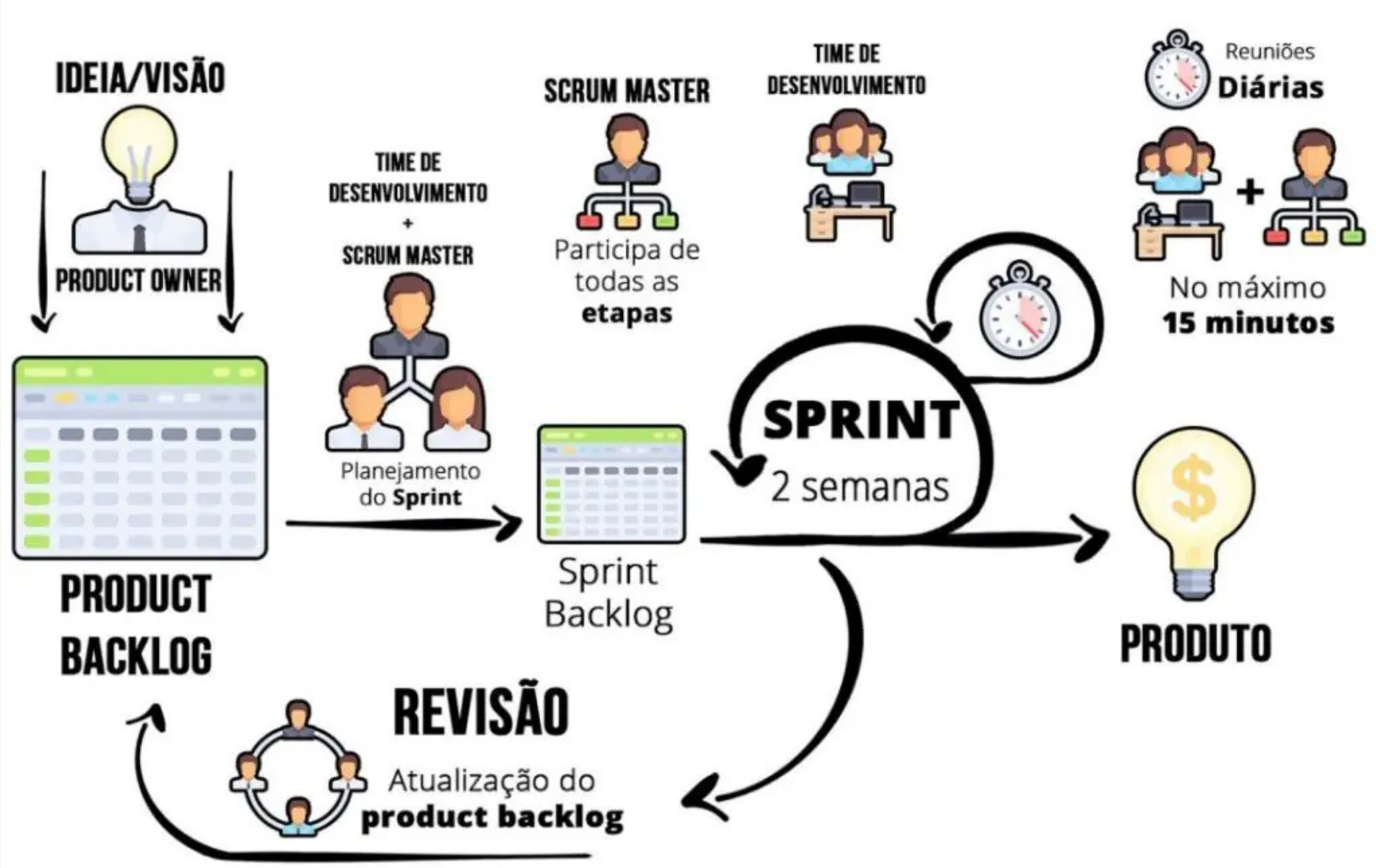


fonte: https://voitto.com.br/blog/artigo/sprint-scrum

CICLO DE SPRINT DO SCRUM



FASES SCRUM

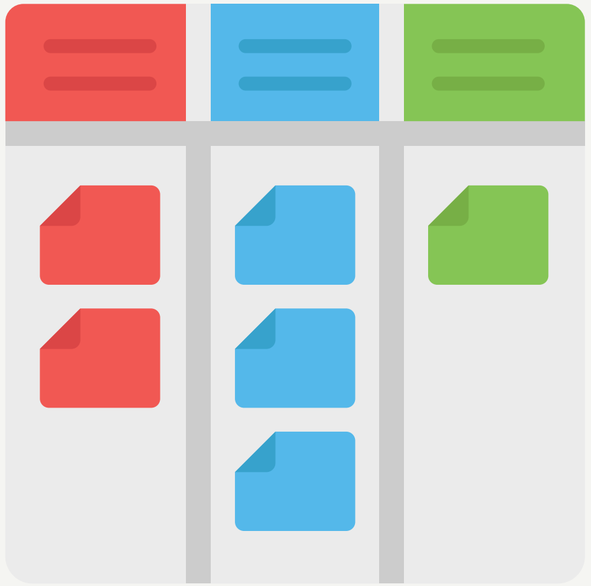

IDEIA/VISÃO
PRODUCT OWNER
PRODUCT BACKLOG
TIME DE DESENVOLVIMENTO + SCRUM MASTER
Planejamento do Sprint
SCRUM MASTER
Participa de todas as etapas
Sprint Backlog
TIME DE DESENVOLVIMENTO
SPRINT
2 semanas
Reuniões Diárias
No máximo 15 minutos
PRODUTO
REVISÃO
Atualização do product backlog

PONTOS POSITIVOS DO SCRUM

1. O produto é decomposto em um conjunto de pedaços gerenciáveis e compreensíveis.
2. Requisitos instáveis não impedem o progresso.
3. O time inteiro tem visibilidade de tudo, e, consequentemente, a comunicação e disposição de seus membros são melhores.
4. Os clientes veem a entrega dos incrementos na hora e obtêm feedback de como o produto funciona.
5. Confiança entre clientes e desenvolvedores é estabelecida, criando uma cultura positiva na qual todos querem o sucesso do projeto.

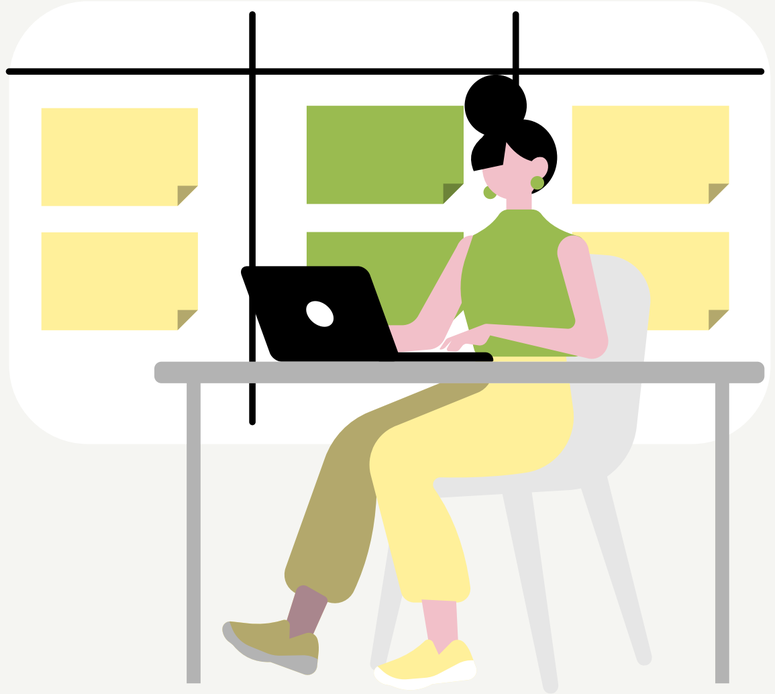


KANBAN



KANBAN

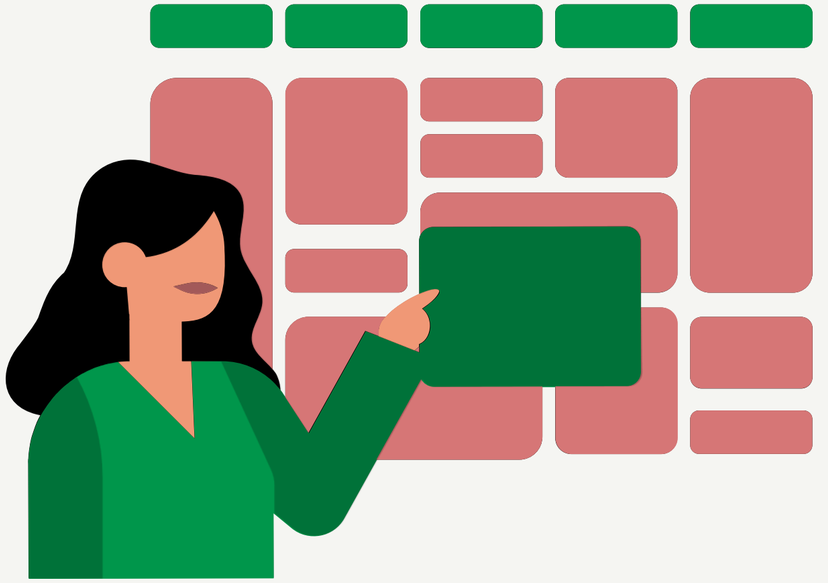
- Kanban surgiu na Toyota e foi adaptado para o desenvolvimento de software por David Anderson.
- Metodologia enxuta focada na gestão de alterações e entrega de serviços, descrevendo métodos para melhorar o fluxo de trabalho.
  - Gestão de alterações define como uma alteração solicitada será integrada.
  - Entrega de serviço é incentivada com foco nas necessidades e expectativas do cliente.
- As reuniões de equipe são realizadas de modo semelhante à metodologia Scrum.



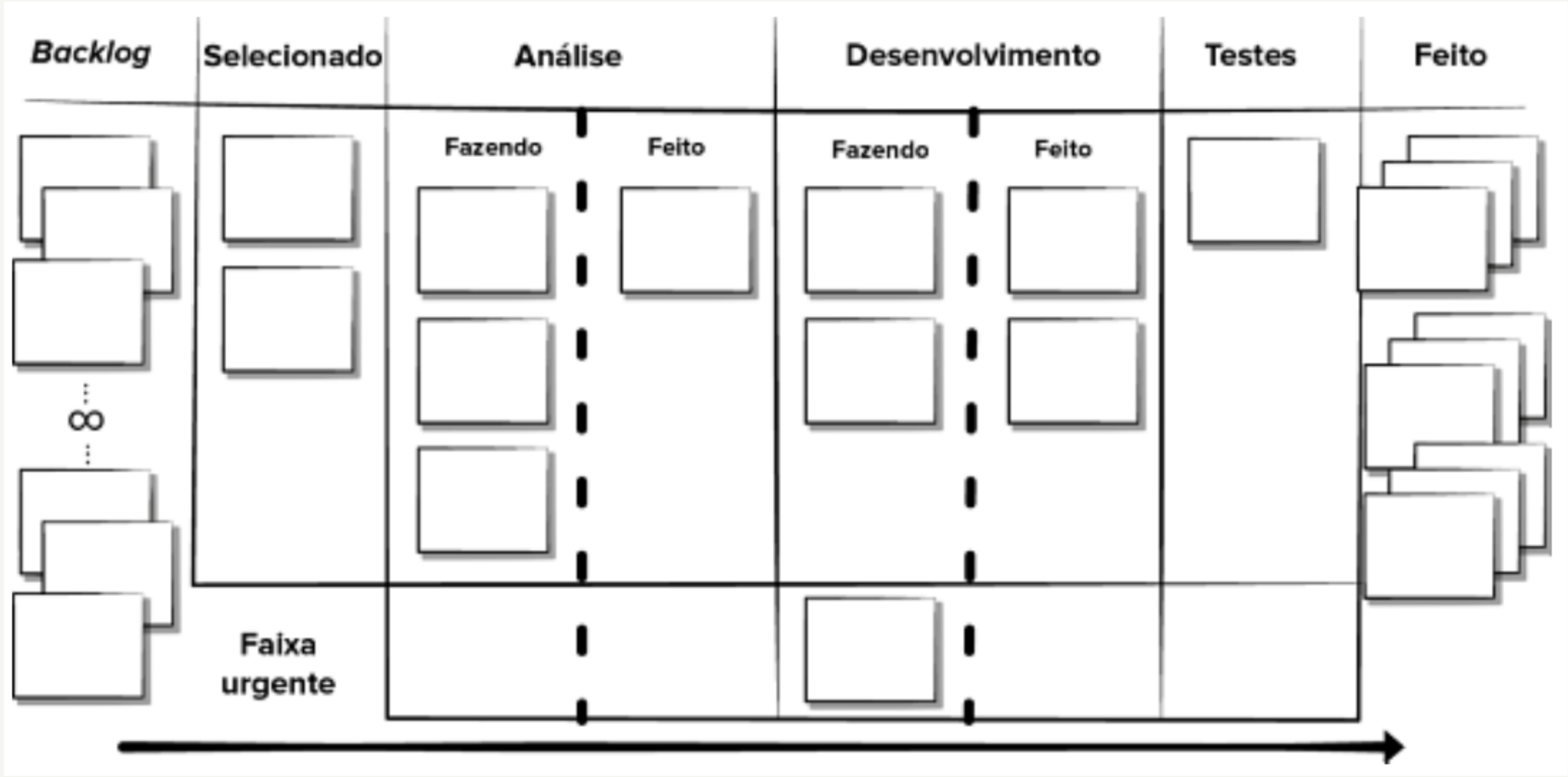
PRÁTICAS FUNDAMENTAIS KANBAN

INSTITUTO FEDERAL
DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
Ceará

1. Visualizar o fluxo de trabalho utilizando um quadro Kanban.
2. Limitar a quantidade de estoque de progresso em um dado momento, incentivando a finalização de uma tarefa antes de iniciar outra.
3. Gerenciar o fluxo de trabalho reduz desperdícios por meio do entendimento do fluxo de valor atual, análise dos pontos de interrupção, definição de mudanças e implementação dessas mudanças.
4. Explicitar as políticas de processo.
5. Focar na melhoria contínua com a criação de ciclos de feedback, nos quais as mudanças são introduzidas e os efeitos sobre o processo são medidos em seguida.
6. Alterar o processo colaborativamente e engajar todos os membros da equipe e outros envolvidos quando necessário.

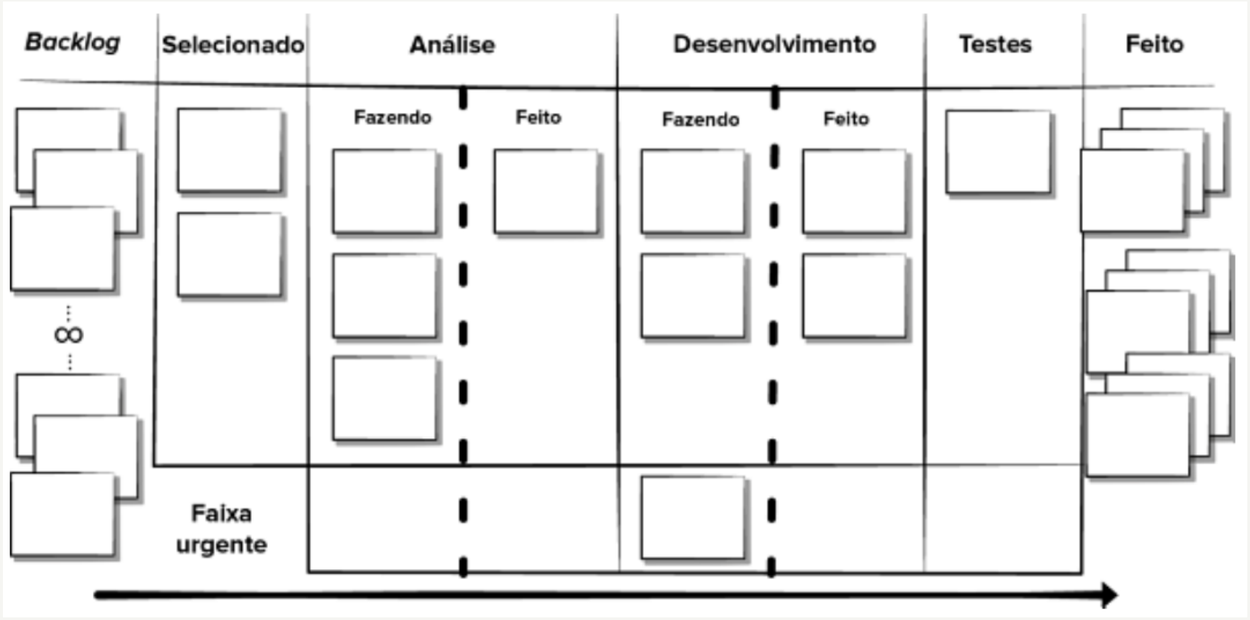


QUADRO KANBAN



(Presman, 2021)

QUADRO KANBAN



- Durante as reuniões diárias (***standup meeting***) é feita a tarefa de “caminhar pelo quadro”.
- A liderança da reunião muda diariamente.
- Membros identificam itens ausentes no quadro que estão sendo trabalhados e os adicionam.
- A equipe tenta fazer com que os itens avancem no quadro, priorizando os itens com alto valor de negócio.
- Equipe identifica obstáculos e riscos à finalização do projeto.

MÉTODOS ÁGEIS - COMPARATIVO

INSTITUTO FEDERAL
DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
Ceará

Metodologia	Prós	Contras
Scrum	• PO define as prioridades. • Equipe se responsabiliza pela tomada de decisões. • Documentação leve. • Apoia atualizações frequentes.	• Difícil de controlar o custo das mudanças. • Pode não ser apropriado para equipes de grande porte. • Exige membros de equipe especializados.
XP	• Enfatiza envolvimento do cliente. • Planos e cronogramas racionais. • Desenvolvedores tem alto comprometimento com o projeto. • Probabilidade de rejeição do produto é menor.	• Tentação de “embalar” o protótipo. • Exige reuniões frequentes sobre custos crescentes. • Pode permitir alterações em excesso. • Dependência de membros qualificados.
Kanban	• Requisitos temporais e orçamentários menores. • Permite entregas mais cedo. • Políticas de processo explicitadas. • Melhoria contínua do processo.	• Depende das habilidades de colaboração da equipe. • Uma análise ruim de negócio pode fracassar o projeto. • Risco de perda de foco.

MÉTODOS ÁGEIS

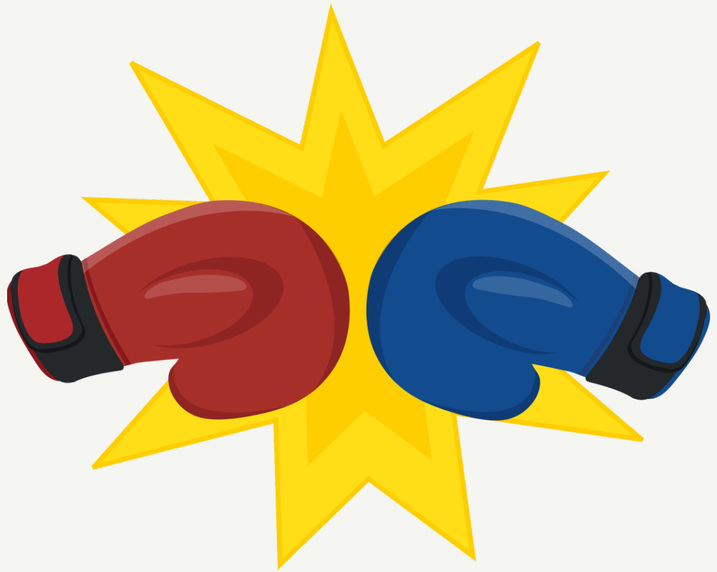
Problemas práticos

INSTITUTO FEDERAL
DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
Ceará

- Em **sistemas grandes e de longa vida útil desenvolvidos para clientes externos**, possíveis problemas:
  - Informalidade incompatível com abordagem tradicional para formalização de contratos.
  - Métodos ágeis são mais adequados para desenvolvimento de softwares novos em vez de manutenção.
  - Métodos são concebidos para pequenas empresas que compartilham uma localização geográfica.
- Na aplicação para **manutenção de software**:
  - Falta de documentação
  - Dificuldade de manter os clientes envolvidos
  - Continuidade do time de desenvolvimento



DESENVOLVIMENTO DIRIGIDO POR PLANOS



DESENVOLVIMENTO ÁGIL

Qual o melhor? Tem espaço para ambos?

Podem ser usados juntos?

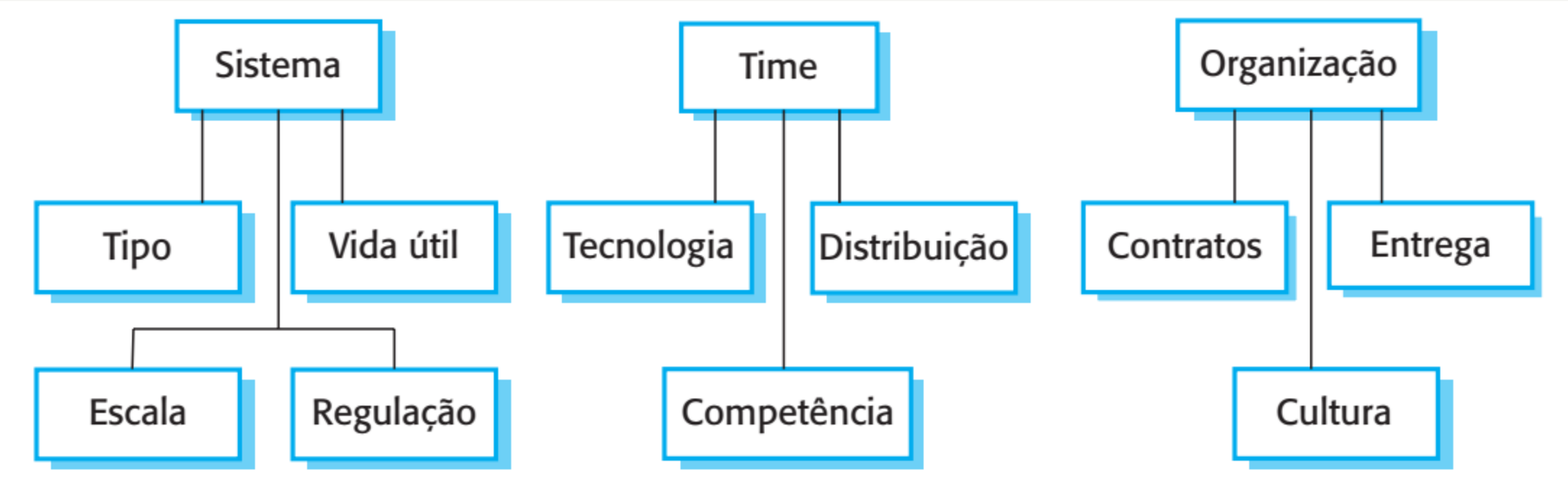
ÁGIL X DIRIGIDO POR PLANOS

- A escolha de um ou outro modelo de desenvolvimento depende de muitos fatores.
- Inicialmente, os adeptos das metodologias ágeis rejeitavam 100% a abordagem tradicional.
- Com o tempo, as empresas foram adaptando os métodos ágeis à sua cultura e modo de trabalhar.
- A maioria dos grandes projetos combinam práticas ágeis e dirigidas por planos.
- Quando usar uma ou outra abordagem depende de alguns fatores: técnicos, humanos e organizacionais.



ÁGIL X DIRIGIDO POR PLANOS

Fatores que influenciam a escolha



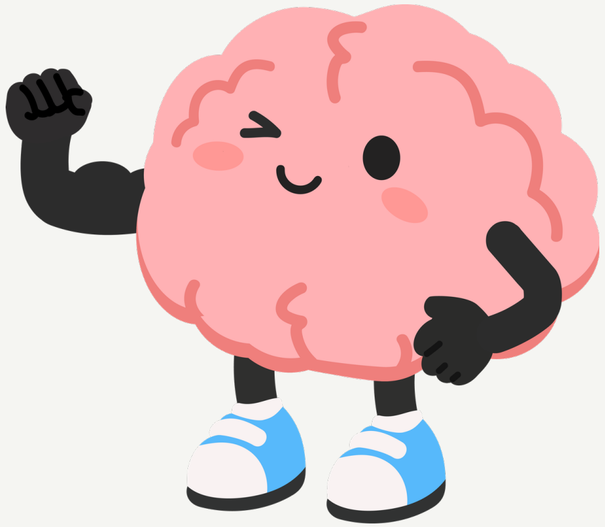
ÁGIL X DIRIGIDO POR PLANOS

Fatores que influenciam a escolha

Sistema	• **Escala do sistema:** sistemas de grande escala exigem grandes equipes e coordenação antecipada. • **Tipo de sistema:** sistemas críticos demandam análises abrangentes antes da codificação. • **Vida útil:** sistemas que serão usados por longo período tendem a exigir maior documentação. • **Regulação/controle externo:** softwares que precisam de aprovação de autoridade reguladora precisam de documentação. • **Tecnologia disponível:** métodos ágeis são bons quando existem boas ferramentas de desenvolvimento e visualização.
Time	• **Nível de competência:** métodos ágeis exigem maior habilidade e responsabilidade (autogestão). • **Distribuição geográfica**: equipes distribuídas tendem a precisar mais de documentação para intermediar a comunicação.
Organização	• **Exigências contratuais:** se for exigido uma especificação e um projeto detaljado antes do início do código, melhor usar abordagem dirigida por planos. • **Entrega e disponibilidade do cliente:** se o cliente estiver disponível para dar feedbacks constantes, é possível abordagem ágil. • **Cultura:** empresas tradicionais tendem a ter resistência a abordagens ágeis.

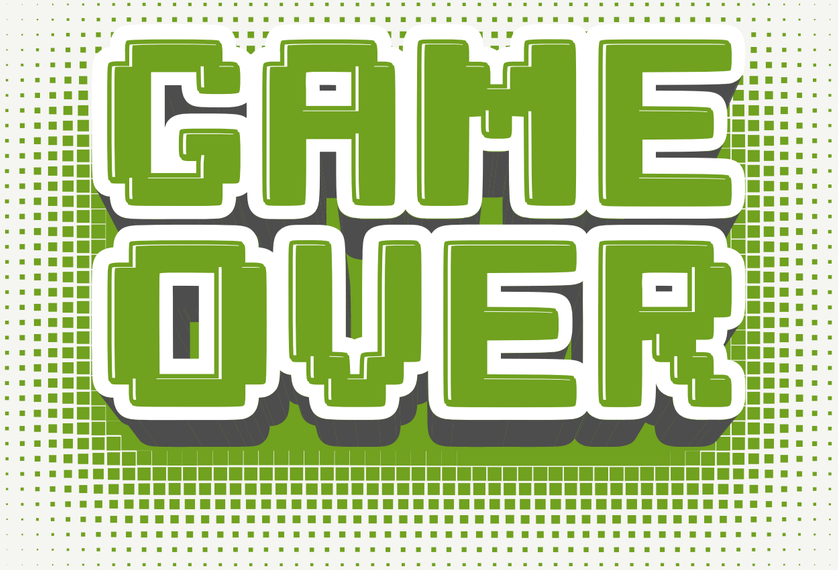
RESUMO

- Modelos de processo representam diferentes formas de organizar as atividades de desenvolvimento de software.
- Os modelos tradicionais, como Cascata, Incremental e Integração e Configuração, apresentam diferentes formas de planejar, desenvolver e entregar o sistema.
- As metodologias ágeis valorizam: entregas frequentes, colaboração entre as pessoas, participação do cliente, feedback contínuo e adaptação às
- XP enfatiza práticas técnicas, como testes, refatoração e programação em pares.
- Scrum organiza o trabalho em ciclos curtos, chamados Sprints.
- Kanban busca visualizar e melhorar continuamente o fluxo de trabalho.
- A escolha da abordagem depende do tipo de sistema, características da equipe e contexto da organização.



POR HOJE É SÓ...

dúvidas, comentários...



FIM

Obrigado pela atenção!

luciano.martins@ifce.edu.br



REFERÊNCIAS

PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de software: uma abordagem profissional. 9. ed. Porto Alegre: McGraw Hill Brasil, 2021. ISBN 978-65-5804-011-8.

SOMMERVILLE, Ian. Engenharia de software. 10. ed. São Paulo: Pearson Prentice Hall, 2018. ISBN 978-85-7936-108-1.